TRABAJO FINAL INTEGRADOR

Sistema de Gestión Logística y Control de Fletes de Caña de Azúcar

Bitácora de trabajo

AUTOR	Maximiliano Rodríguez
TIPO DE TRABAJO	Trabajo Final Integrador (TFI)
DOCENTE / TUTOR	*[completar]*
CÁTEDRA	*[completar]*
INSTITUCIÓN	*[completar]*
PERÍODO DOCUMENTADO	06/09/2026 – 02/11/2026
ÚLTIMA ACTUALIZACIÓN	02/11/2026

Contenido

1. Resumen del proyecto 3
1.1 Problema y propuesta 3
1.2 Alcance funcional 3
1.3 Roles del sistema 3
2. Registro resumido 4
3. Entradas por semana 5
3.1 Semana del 31/08 al 06/09/2026 — Definición del tema y estructura inicial 5
3.2 Semana del 19/10 al 25/10/2026 — Diseño del modelo relacional y script SQL 5
3.3 Semana del 26/10 al 01/11/2026 — Base de datos, backend y frontend 6
3.3.1 Base SQLite y normalización en 3FN 6
3.3.2 Backend FastAPI con autenticación JWT, roles y tests 6
3.3.3 Frontend React con login por roles y CRUD de localidades 7
3.4 Semana del 02/11 al 08/11/2026 — Puesta en marcha y acceso desde otra computadora 8
4. Estado actual del sistema 9
5. Próximos pasos y pendientes 10
6. Anexos 11
6.1 Anexo A — Arquitectura del sistema 11
6.2 Anexo B — Modelo de datos 11
6.3 Anexo C — Endpoints y permisos 12
6.4 Anexo D — Pruebas automatizadas 12
6.5 Anexo E — Glosario 13
6.6 Anexo F — Historial de commits 13
7. Plantilla para las próximas entradas 14

> **Cómo leer este documento.** Las fechas, actividades y evidencias surgen del historial de Git del repositorio y del código efectivamente implementado. Las horas son **estimaciones** basadas en el alcance de cada etapa: deben ajustarse con el registro real del autor. Los campos marcados `[completar]` requieren información que no queda registrada en el repositorio (institución, docentes, reuniones o correcciones). Las fechas se expresan como DD/MM/AAAA y las horas como estimaciones aproximadas.

1. Resumen del proyecto

1.1 Problema y propuesta

Durante la zafra, el transporte de caña de azúcar desde los frentes de cosecha (fincas) hasta las plantas de procesamiento (ingenios) se documenta históricamente con "vales de báscula" en papel. Ese circuito genera cuatro problemas concretos:

- Pérdida de información y errores en la carga manual de datos.
- Demoras operativas para conocer el peso neto real de la materia prima entregada.
- Dificultad y retrasos en la liquidación de los pagos (fletes) a transportistas y choferes.
- Falta de trazabilidad exacta sobre el origen (finca/productor) y el destino (ingenio) de cada viaje.

La propuesta consiste en una API REST centralizada que reemplaza ese manejo manual: registra el circuito operativo completo y calcula en tiempo real las métricas financieras y operativas, sin almacenar datos redundantes.

1.2 Alcance funcional

1. **Gestión de origen y destino:** alta, baja y modificación de fincas y productores; registro de los ingenios receptores.
2. **Administración de flota y personal:** control del parque automotor por patente y padrón de choferes con múltiples teléfonos de contacto.
3. **Control transaccional de báscula:** emisión digital del vale de transporte, registro del peso bruto (ingreso cargado al ingenio) y de la tara (salida vacío).
4. **Motor de liquidación automática:** cálculo del peso neto (bruto − tara) y del importe del flete según la tarifa por tonelada pactada.

1.3 Roles del sistema

Rol	Quién es	Permisos
`ADMIN`	Jefes y administración	Lee y escribe todos los catálogos, gestiona usuarios y administra cualquier viaje.
`CHOFER`	Choferes	Lee los catálogos necesarios; solo registra, ve y edita **sus propios** viajes (peso bruto, tara y tarifa). No puede eliminarlos ni ver los de otros choferes.

2. Registro resumido

Semana	Etapa	Horas estimadas	Evidencia
31/08 – 06/09/2026	Definición del tema y estructura inicial	≈ 6 h	`cda263f`, `41d1359`
19/10 – 25/10/2026	Diseño del modelo relacional y script SQL	≈ 8 h	`3350b91`
26/10 – 01/11/2026	Base SQLite y normalización en 3FN	≈ 10 h	`cc9070e`, `af419b0`
26/10 – 01/11/2026	Backend FastAPI (autenticación, roles, CRUD y tests)	≈ 20 h	`209d82e`
26/10 – 01/11/2026	Frontend React (login por roles y localidades)	≈ 12 h	`1345794`
02/11 – 08/11/2026	Puesta en marcha en red local	≈ 3 h	Prueba manual
Total		**≈ 59 h**	

3. Entradas por semana

3.1 Semana del 31/08 al 06/09/2026 — Definición del tema y estructura inicial

Horas estimadas: ≈ 6 h

Objetivo: elegir el tema del TFI y dejar armado el esqueleto del repositorio.

Actividades:

- Análisis del problema y de sus consecuencias operativas: pérdida de información, errores de carga manual, demoras en el peso neto y en la liquidación de fletes, y falta de trazabilidad.
- Definición del alcance en cuatro módulos: origen y destino, flota y personal, control de báscula y liquidación automática.
- Elección del stack: Python + FastAPI, SQLAlchemy 2.0, Pydantic v2, SQLite y autenticación JWT con Argon2.
- Creación del repositorio y del `Readme.md` con introducción, contexto, objetivos y arquitectura.
- Organización inicial en carpetas `backend/` y `frontend/`.

Decisiones técnicas:

- Separar backend y frontend desde el inicio para desarrollar y probar la API de forma independiente.
- Documentar la arquitectura y el alcance en el `Readme.md` antes de escribir código, para usarlo como guía y como entregable.

Dificultades: ninguna técnica registrada en esta etapa; el esfuerzo estuvo en acotar el alcance del sistema.

Próximos pasos: diseñar el modelo de datos.

Evidencia: commits `cda263f` ("Commit inicial: Estructura base del proyecto") y `41d1359` ("Readme"), ambos del 06/09/2026.

3.2 Semana del 19/10 al 25/10/2026 — Diseño del modelo relacional y script SQL

Horas estimadas: ≈ 8 h

Objetivo: pasar del alcance funcional a un modelo de datos concreto y ejecutable.

Actividades:

- Identificación de entidades y relaciones: localidad/departamento, finca, productor, ingenio, camión, chofer, teléfono de chofer, viaje y usuario.
- Definición de claves primarias, claves foráneas y restricciones de integridad.
- Escritura del script de creación de la base (`logistica_agro.sql`), con tablas, restricciones y datos iniciales.
- Revisión de la cardinalidad de las relaciones: por ejemplo, un productor puede estar asociado a varias fincas y un chofer tiene varios teléfonos.

Decisiones técnicas:

- Modelo relacional en Tercera Forma Normal (3FN): los datos derivados (peso neto e importe del flete) **no** se almacenan, se calculan en la API.

- Uso de claves naturales cuando aportan significado al negocio: patente de camión, DNI de chofer, CUIT de productor y número de vale.

Dificultades:

- Ajustar el modelo para evitar redundancia entre localidad y departamento; se resolvió con la normalización de la semana siguiente.

Próximos pasos: llevar el script a una base utilizable por la API y cargar datos de prueba.

Evidencia: commit `3350b91` ("Agrega script SQL de creación de la base de datos logistica_agro"), 19/10/2026.

3.3 Semana del 26/10 al 01/11/2026 — Base de datos, backend y frontend

Semana de mayor avance: se pasó del diseño a un sistema ejecutable de punta a punta. Por su extensión, la entrada se divide en tres bloques.

Horas estimadas: ≈ 42 h en total (10 h de base de datos + 20 h de backend + 12 h de frontend).

3.3.1 Base SQLite y normalización en 3FN

Objetivo: que el esquema pudiera ejecutarse localmente sin depender de un motor externo y sin redundancia.

Actividades:

- Migración del esquema a SQLite con restricciones `CHECK`, `UNIQUE` y claves foráneas.
- Creación de `seed.sql` con datos de prueba: localidades, fincas, productores, camiones, choferes e ingenios.
- Incorporación de `.gitignore` para no versionar la base ni el entorno virtual.
- Normalización de localidades y departamentos en 3FN, con su router, esquemas y tests.

Decisiones técnicas:

- Activar `PRAGMA foreign_keys = ON` en cada conexión (`backend/app/database.py`), porque SQLite no valida las claves foráneas por defecto.
- Mantener la URL de la base configurable por `DATABASE_URL`, con SQLite como valor por defecto.
- Separar la estructura (`logistica_agro.sql`) de los datos de prueba (`seed.sql`).

Dificultades:

- SQLite no aplica integridad referencial de forma automática: se detectó y se resolvió con el `PRAGMA` en el evento de conexión.

Evidencia: commits `cc9070e` ("Migra la base de datos a SQLite con restricciones, datos de prueba y gitignore") y `af419b0` ("Normaliza localidades y departamentos en 3FN"), 26/10/2026.

3.3.2 Backend FastAPI con autenticación JWT, roles y tests

Objetivo: exponer la API REST completa con seguridad por roles.

Actividades:

- Modelos SQLAlchemy 2.0 y esquemas Pydantic v2 para todas las entidades.

- Routers por entidad: `auth`, `localidades`, `fincas`, `productos`, `camiones`, `choferes`, `ingenios`, `viajes` y `usuarios`.
- Autenticación con OAuth2 password flow, JWT firmado y contraseñas hasheadas con Argon2.
- Dependencias de autorización: `requiere_admin` y restricciones por chofer.
- Reglas de negocio: cálculo de peso neto y flete, rechazo de bruto menor que tara, vale duplicado, productor no asociado a la finca y protección de viajes ajenos.
- Script `seed_usuarios.py` para crear las cuentas iniciales.
- Suite de tests de integración con pytest (25 casos) sobre autenticación, permisos, CRUD, cálculos y filtros.
- Configuración de `pytest` y `ruff` en `pyproject.toml`.

Decisiones técnicas:

- Los cálculos de peso neto y flete se resuelven en la API y no se persisten, para no duplicar información.
- Un chofer que intenta acceder al viaje de otro recibe `404` en lugar de `403`, para no revelar la existencia del recurso.
- Errores de integridad traducidos a respuestas HTTP claras: `409` al borrar entidades con dependencias y `404` centralizado en `utils.py`.
- CORS abierto para permitir el desarrollo del frontend en otro puerto.

Dificultades:

- Coordinar autenticación, roles y filtrado por pertenencia en un mismo endpoint sin duplicar lógica; se resolvió con dependencias reutilizables y helpers de consulta.
- Probar permisos negativos (chofer editando un viaje ajeno, administrador desactivándose a sí mismo) requirió armar fixtures de usuarios y datos en `conftest.py`.

Evidencia: commit `209d82e` ("Agrega backend FastAPI con autenticación JWT, roles y organiza backend/frontend"), 26/10/2026.

3.3.3 Frontend React con login por roles y CRUD de localidades

Objetivo: contar con una interfaz mínima que consuma la API y valide el circuito de autenticación.

Actividades:

- Proyecto React 18 + TypeScript + Vite + Tailwind.
- Cliente Axios con token en `localStorage`, interceptor de `401` y cierre de sesión automático.
- Contexto de autenticación, rutas protegidas por rol y layout general.
- Pantalla de login, redirección según rol y pantalla 404.
- CRUD de localidades como primera pantalla funcional.
- Tests de rutas protegidas y utilidades de formato.
- Documento `PROMPT_FRONTEND.md` con la especificación de las pantallas restantes.

Decisiones técnicas:

- El rol se toma siempre de `/auth/me`, no del token, para reflejar cambios de permisos.
- La URL de la API se configura con `VITE_API_URL`, con `http://127.0.0.1:8000/api/v1` como valor por defecto.

- Las pantallas no implementadas usan un componente `Placeholder` para dejar visible el alcance pendiente.

Dificultades:

- Mantener sincronizado el estado de sesión entre el almacenamiento local, el contexto y las rutas protegidas.

Próximos pasos: implementar las pantallas restantes (viajes, fincas, productores, camiones, choferes, ingenios y usuarios).

Evidencia: commit `1345794` ("Agrega frontend React con login por roles y CRUD de localidades"), 26/10/2026.

3.4 Semana del 02/11 al 08/11/2026 — Puesta en marcha y acceso desde otra computadora

Horas estimadas: ≈ 3 h

Objetivo: verificar el sistema en un escenario de uso real con dos equipos en la misma red.

Actividades:

- Ejecución del backend y del frontend en el equipo de desarrollo.
- Prueba de login desde una segunda computadora.
- Diagnóstico de los tres motivos por los que el acceso no funcionaba: la URL `127.0.0.1` apunta al propio equipo cliente, el backend y Vite escuchaban solo en localhost, y el firewall no tenía reglas para los puertos 8000 y 5173.
- Verificación de que cada equipo tiene su propia base SQLite y, por lo tanto, necesita ejecutar `python -m app.seed_usuarios` para tener las cuentas iniciales.

Decisiones técnicas:

- Para el escenario de dos equipos se evaluaron dos alternativas: exponer ambos servidores en `0.0.0.0` con la IP de la red local, o usar un único backend compartido y que el segundo equipo consuma `http://<IP-del-servidor>:8000/api/v1`.
- Se definió que un backend único es preferible a mantener dos bases independientes, porque evita duplicar datos y usuarios.

Dificultades:

- Confusión inicial entre "no funciona el login" y "no existe el usuario en esta base": el problema era que la segunda computadora tenía una base vacía, no un error de credenciales.
- La URL `127.0.0.1` en el `.env` del frontend impide el acceso desde otro equipo; queda pendiente parametrizarla por entorno.

Próximos pasos:

- Definir una estrategia de despliegue (servidor propio, túnel o hosting gratuito con base persistente).
- Configurar `SECRET_KEY` y cambiar las contraseñas por defecto antes de publicar.

Evidencia: prueba manual del 02/11/2026, sin commit asociado.

---

4. Estado actual del sistema

Funciona:

- API REST con documentación automática en `/docs` y `/redoc`.
- Login con JWT, roles `ADMIN` y `CHOFER`, y gestión de usuarios.
- CRUD completo de localidades, fincas, productos, camiones, choferes, ingenios y viajes.
- Cálculo de peso neto y flete, filtros de viajes y reglas de pertenencia por chofer.
- Suite de 25 tests de integración del backend.
- Frontend con login por roles, rutas protegidas y CRUD de localidades.

Pendiente:

- Pantallas de viajes, fincas, productores, camiones, choferes, ingenios y usuarios en el frontend.
- Dashboard y pantalla de perfil del chofer.
- Despliegue y configuración de secretos de producción.
- Estrategia de base compartida entre equipos (backend único o migración a PostgreSQL).

---

5. Próximos pasos y pendientes

- ☐ Implementar las pantallas restantes del frontend.
- ☐ Definir y documentar el despliegue (hosting, base de datos y variables de entorno).
- ☐ Configurar `SECRET_KEY` por variable de entorno y cambiar `admin123` / `chofer123`.
- ☐ Resolver el acceso multiusuario: backend único o PostgreSQL mediante `DATABASE_URL`.
- ☐ Incorporar una entrada semanal a esta bitácora.
- ☐ Commitear o eliminar el `logistica_agro.sql` duplicado que quedó sin trackear en la raíz del repositorio.

---

6. Anexos

6.1 Anexo A — Arquitectura del sistema

El sistema se organiza en tres capas, de modo que cada una pueda modificarse sin romper las demás:

Capa	Tecnología	Responsabilidad
Presentación	React 18 + TypeScript + Vite + Tailwind	Interfaz de usuario, rutas por rol y consumo de la API mediante Axios.
Aplicación	FastAPI + Pydantic v2	Endpoints REST, validación de entrada, autenticación JWT y reglas de negocio.
Persistencia	SQLAlchemy 2.0 + SQLite 3	Modelo relacional, integridad referencial y consultas.

Flujo de una petición. El cliente envía la petición con el token JWT en el encabezado `Authorization: Bearer <token>`. FastAPI valida el token, resuelve el usuario y su rol mediante dependencias (`UsuarioActual`, `AdminActual`, `ChoferActual`), y deriva al router correspondiente. El router valida el cuerpo con un esquema Pydantic, aplica las reglas del negocio y opera sobre la base a través de una sesión de SQLAlchemy. La respuesta se serializa con otro esquema Pydantic, que es el que calcula propiedades derivadas como el peso neto y el flete.

Organización del backend. `main.py` registra los routers bajo el prefijo `/api/v1` y configura CORS. `models.py` contiene las tablas; `schemas.py`, los contratos de entrada y salida; `auth.py`, el hash de contraseñas, la emisión y validación de JWT y las dependencias de rol; `utils.py` centraliza el `404` y la traducción de errores de integridad; `routers/` contiene un módulo por entidad.

6.2 Anexo B — Modelo de datos

Entidad	Clave	Campos principales	Relaciones
`LOCALIDADES`	`ID_Localidad`	Nombre, Departamento	Una localidad agrupa muchas fincas. Evita repetir el departamento.
`FINCAS`	`ID_Finca`	Nombre, `ID_Localidad`	Pertenece a una localidad; se asocia con productores y origina viajes.
`PRODUCTOS`	`Producto_CUIT`	Nombre	Se asocia a fincas y participa en viajes.
`CAMIONES`	`Camion_Patente`	Patente	Realiza viajes.
`CHOFERES`	`Chofer_DNI`	Nombre	Tiene teléfonos y realiza viajes.
`TELEFONOS_CHOFER`	(`Chofer_DNI`, `Telefono`)	Teléfono	Varios teléfonos por chofer; se eliminan en cascada.
`INGENIOS`	`ID_Ingenio`	Nombre único	Recibe viajes.
`VIAJES`	`ID_Viaje`	Vale único, finca, productor, camión, chofer, ingenio, bruto, tara y tarifa	Une todas las entidades del circuito.
`USUARIOS`	`ID_Usuario`	Usuario, hash, rol y `Chofer_DNI` opcional	Un usuario `CHOFER` se vincula a un chofer; un `ADMIN` no.

Decisiones de integridad. La tabla `VIAJES` exige que el productor esté asociado a la finca de origen mediante una clave foránea compuesta contra `FINCA_PRODUCTO`. Además, restringe `Peso_Bruto_Kg >= Tara_Kg`, `Tara_Kg`

`>= 0` y `Tarifa_Ton >= 0`, y el número de vale es único. El peso neto y el flete no se guardan: son propiedades calculadas (`models.py:158-167`) y se exponen en la respuesta.

6.3 Anexo C — Endpoints y permisos

Todos los endpoints se montan bajo `/api/v1`. La lectura de catálogos está disponible para ambos roles; la escritura, solo para `ADMIN`.

Recurso	Rutas	Lectura	Escritura
Autenticación	`POST /auth/login`, `GET /auth/me`	Público / autenticado	—
Localidades	`GET`, `GET /{id}`, `POST`, `PUT /{id}`, `DELETE /{id}`	Ambos roles	`ADMIN`
Fincas	`GET`, `GET /{id}`, `POST`, `PUT /{id}`, `DELETE /{id}`	Ambos roles	`ADMIN`
Productores de una finca	`POST /fincas/{id}/productores/{cuit}`, `DELETE` ídem	Ambos roles	`ADMIN`
Productos	`GET`, `GET /{cuit}`, `POST`, `PUT /{cuit}`, `DELETE /{cuit}`	Ambos roles	`ADMIN`
Camiones	`GET`, `GET /{patente}`, `POST`, `PUT /{patente}`, `DELETE /{patente}`	Ambos roles	`ADMIN`
Choferes	`GET`, `GET /{dni}`, `POST`, `PUT /{dni}`, `DELETE /{dni}`	Ambos roles	`ADMIN`
Ingenios	`GET`, `GET /{id}`, `POST`, `PUT /{id}`, `DELETE /{id}`	Ambos roles	`ADMIN`
Viajes	`GET`, `GET /mis-viajes`, `GET /{id}`, `POST`, `PUT /{id}`, `DELETE /{id}`	`ADMIN` ve todos; `CHOFER`, solo los propios	`ADMIN` cualquiera; `CHOFER` solo los propios y sin eliminar
Usuarios	`GET`, `POST`, `PUT /{id}`	`ADMIN`	`ADMIN`

Detalles de los viajes. El listado admite paginación (`skip`, `limit`) y filtros por finca, ingenio, chofer, camión y productor. Un chofer que intenta ver o editar un viaje ajeno recibe `404` para no revelar su existencia, y no puede registrar viajes a nombre de otro chofer ni eliminarlos.

6.4 Anexo D — Pruebas automatizadas

Los tests se ejecutan con `pytest` sobre una base SQLite temporal creada en `conftest.py`; cada caso arranca con el esquema vacío y usuarios de prueba. Son 25 casos agrupados así:

Área	Casos	Qué verifica
Salud y autenticación	4	`/` y `/health`, rechazo sin token, credenciales incorrectas y datos de `/auth/me`.
Gestión de usuarios	3	Alta y listado por `ADMIN`, prohibición para `CHOFER`, cambio de contraseña y baja lógica.
Catálogos y relaciones	6	CRUD de localidades y fincas, integridad referencial, asociación productor–finca, teléfonos y lectura por `CHOFER`.
Viajes: cálculo, filtros y borrado	5	Peso neto y flete, actualización de cálculos, filtros, viaje inexistente y protección de fincas con viajes.

Viajes: validaciones	3	Bruto menor que tara, vale duplicado y productor no asociado a la finca.
Permisos del chofer	4	Registro propio, prohibición de registrar para otro, visibilidad/edición solo de los propios y prohibición de eliminar.

6.5 Anexo E — Glosario

Término	Definición
Zafra	Temporada de cosecha y molienda de la caña de azúcar.
Vale de báscula	Comprobante del pesaje de un camión que ingresa cargado y egresa vacío.
Peso bruto	Peso del camión cargado al ingresar al ingenio.
Tara	Peso del camión vacío.
Peso neto	Carga útil real: peso bruto menos tara.
Flete	Importe a pagar al transportista: toneladas netas por la tarifa pactada.
3FN	Tercera Forma Normal: evita dependencias transitivas y datos redundantes.
JWT	Token firmado que identifica al usuario en cada petición.
ORM	Capa que traduce objetos del código a tablas de la base de datos.
CRUD	Operaciones de alta, lectura, actualización y baja.
SPA	Aplicación web de una sola página que se actualiza sin recargar el navegador.

6.6 Anexo F — Historial de commits

Commit	Fecha	Descripción
`cda263f`	06/09/2026	Commit inicial: estructura base del proyecto
`41d1359`	06/09/2026	Readme
`3350b91`	19/10/2026	Agrega script SQL de creación de la base de datos `logistica_agro`
`cc9070e`	26/10/2026	Migra la base de datos a SQLite con restricciones, datos de prueba y gitignore
`209d82e`	26/10/2026	Agrega backend FastAPI con autenticación JWT, roles y organiza backend/frontend
`af419b0`	26/10/2026	Normaliza localidades y departamentos en 3FN
`1345794`	26/10/2026	Agrega frontend React con login por roles y CRUD de localidades

7. Plantilla para las próximas entradas

```
### Semana del DD/MM al DD/MM/AAAA — Título de la etapa

**Horas estimadas:** ≈ N h

**Objetivo:**

**Actividades:**
-

**Decisiones técnicas:**
-

**Dificultades y cómo se resolvieron:**
-

**Próximos pasos:**

**Evidencia:** commits `xxxxxxx` (DD/MM/AAAA)
```

> Las horas de la sección 2 son estimaciones y no surgen de Git: el repositorio no registra tiempo de trabajo. Ajustarlas antes de la entrega.